Interfaces funcionales. Expresiones lambda en Java

Esta obra, cuyo autor es Marcos Martínez Rus, ha sido publicada bajo una
licencia de Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional

ÍNDICE

1. Introducción....3

2. Expresiones lambda....3

3. Clase Persona....5

4. Funciones. Function<T,R>....7

4.1 Implementación mediante clase....7

4.2 Implementación sin uso de clase independiente....8

5. Predicado. Predicate<T>....8

5.1 Implementación mediante clase....9

5.2 Implementación sin uso de clase independiente....9

6. Consumidor. Consumer<T>....10

6.1 Implementación mediante clase....11

6.2 Implementación sin uso de clase independiente....11

7. Proveedor. Supplier<R>....11

7.1 Implementación mediante clase....12

7.2 Implementación sin uso de clase independiente....12

8. Comparadores haciendo uso de lambdas....13

9. Métodos útiles....13

10. Buenas prácticas....14

11. Aspectos a tener en cuenta....15

1. Introducción

Una interfaz funcional es una interfaz con un y sólo un método abstracto. La declaración es exactamente igual que las interfaces normales con dos características adicionales:

- Tiene un único método abstracto.
- De manera opcional puede estar anotada como `@FunctionalInterface`.
- El motivo de que la interfaz funcional tenga un único método abstracto es que permitirá proveer la lógica de implementación de dicho método con una expresión lambda si se desea, reduciendo el código necesario.

Durante el presente tema, nos vamos a centrar en el estudio de las siguientes cuatro interfaces funcionales.

- Funciones (`Function<T,R>`).
- Predicados (`Predicate<T>`).
- Consumidores (`Consumer<T>`).
- Proveedores (`Supplier<R>`).

2. Expresiones lambda

Java 8 trajo una nueva y poderosa mejora sintáctica en forma de **expresiones lambda** (programación funcional). Una lambda es una función anónima que podemos manejar como si fuese un parámetro cualquiera. Por ejemplo, podemos pasarla o devolverla desde un método. El hecho de vincularse al concepto de "función" implica que "hace algo" con los datos (no es un parámetro que almacene información sino que la gestiona).

De esta manera, hasta ahora estamos acostumbrados en los métodos a pasarles información o devolverla (cadenas de texto, números, booleanos, …). Con las expresiones lambda lo que podremos es pasar o devolver "comportamiento": qué hacer.

Antes de Java 8, normalmente creábamos una clase para cada caso en el que necesitábamos encapsular una funcionalidad. Ésto implicó una gran cantidad de código repetitivo innecesario. Por ejemplo, si se necesitaban cinco criterios de comparación de una clase, se necesitaba implementar cinco clases `Comparator`.

Las expresiones lambda son funciones anónimas, es decir, funciones que no necesitan una clase. Su sintaxis básica se detalla a continuación:

```
(parámetros) -> {código-expresión-lambda}
```

La estructura es la siguiente:

a) **Parámetros**: datos de entrada a la función lambda.
   - Cuando se tiene un único parámetro no es necesario utilizar los paréntesis.
   - Cuando no se tienen parámetros, o cuando se tienen dos o más, es necesario utilizar paréntesis.

b) El **operador lambda (->)** separa la declaración de parámetros de la declaración del cuerpo de la función.

c) **Código de la expresión lambda**:
   - Cuando el cuerpo de la expresión lambda tiene una única línea no es necesario utilizar las llaves y no necesitamos especificar `return` en el caso de tener que devolver valores.
   - Cuando el cuerpo de la expresión lambda tiene más de una línea se hace necesario utilizar las llaves y es necesario incluir `return` en el caso de que la función deba devolver un valor. Como buena práctica, lo recomendable es que la expresión lambda no contenga múltiples sentencias.

Algunos ejemplos de expresiones lambda pueden ser:

Expresión lambda	Funcionalidad
z -> z + 2	Recibe un parámetro y le suma 2
() -> System.out.println("Hola mundo")	Muestra, por consola, el saludo
(double base, double altura) -> { return base * altura; }	Devuelve el área de un rectángulo

Existe mucha información en Internet sobre expresiones lambda e interfaces funcionales. Durante el presente tema, vamos a centrarnos en los aspectos más importantes pero es recomendable leer artículos sobre estos conceptos para conseguir mejorar como profesionales en el desarrollo de software.

3. Clase Persona

Para las explicaciones de este tema, vamos a utilizar la siguiente clase que modela los datos de una persona.

```
public class Persona {
	private int dni;
	private String nombre;
	private String apellidos;
	private LocalDate fechaNacimiento;

	public Persona(int dni, String nombre, String apellidos, LocalDate fechaNacimiento)
	{
		super();
		this.dni = dni;
		this.nombre = nombre;
		this.apellidos = apellidos;
		this.fechaNacimiento = fechaNacimiento;
	}

	public int getDni() {
		return dni;
	}

	public void setDni(int dni) {
		this.dni = dni;
	}

	public String getNombre() {
		return nombre;
	}
```

```java
public void setNombre(String nombre) {
	this.nombre = nombre;
}

public String getApellidos() {
	return apellidos;
}

public void setApellidos(String apellidos) {
	this.apellidos = apellidos;
}

public LocalDate getFechaNacimiento() {
	return fechaNacimiento;
}

public void setFechaNacimiento(LocalDate fechaNac) {
	this.fechaNacimiento = fechaNac;
}

public int edad() {
	return (int) ChronoUnit.YEARS.between(getFechaNacimiento(), LocalDate.now());
}

@Override
public String toString() {
	return "Persona [dni=" + dni + ", nombre=" + nombre + ", apellidos=" +
			apellidos + ", fechaNacimiento=" + fechaNacimiento + "]";
}

@Override
public boolean equals(Object obj) {
	if (this == obj) return true;
	if (obj == null) return false;
	if (getClass() != obj.getClass()) return false;

	Persona other = (Persona) obj;
	return Objects.equals(apellidos, other.apellidos) && dni == other.dni
			&& Objects.equals(fechaNacimiento, other.fechaNacimiento)
			&& Objects.equals(nombre, other.nombre);
}
```

CC BY NC SA

```
	@Override
	public int hashCode() {
		return Objects.hash(apellidos, dni, fechaNacimiento, nombre);
	}
}
```

Para el desarrollo del tema y la realización de los ejemplos vamos a suponer que tenemos la siguiente lista de Personas.

```
	List<Persona> personas = new ArrayList<Persona>();
	personas.add(new Persona(11, "Paco", "Gento", LocalDate.of(1933, 10, 21)));
	personas.add(new Persona(9, "Alfredo", "Di Stéfano", LocalDate.of(1926, 7, 4)));
	personas.add(new Persona(5, "Zinedine", "Zidane", LocalDate.of(1972, 6, 23)));
	personas.add(new Persona(10, "Luka", "Modric", LocalDate.of(1985, 9, 9)));
	personas.add(new Persona(7, "Raúl", "González Blanco", LocalDate.of(1977, 6, 27)));
```

4. Funciones. Function<T,R>

Es el caso más simple y general de una lambda. Es una interfaz funcional que **recibe un valor y devuelve otro valor**. Para implementarlas debemos usar la interfaz `Function<T,R>`, donde `T` indica el tipo del dato de entrada y `R` el tipo de dato de retorno. Para ejecutar la función debemos hacer uso de su método **`apply()`**.

4.1 Implementación mediante clase

Supongamos que dada una `Persona`, queremos devolver su nombre completo. Lo podemos realizar de la siguiente forma.

```
public class NombreCompleto implements Function<Persona, String> {

	@Override
	public String apply(Persona p) {
		return p.getNombre() + " " + p.getApellidos();
	}
}
```

Para hacer uso de dicha función, podemos implementar lo siguiente. En ella, mostramos el nombre completo de la primera persona de nuestra lista.

```
Function<Persona, String> nombreCompleto = new NombreCompleto();
String nombreApellidos = nombreCompleto.apply(personas.get(0));

System.out.println(nombreApellidos);
```

4.2 Implementación sin uso de clase independiente

También podemos crear una función sin necesidad de generar una clase nueva. A continuación, se muestra un ejemplo de declaración de una función que permite, recibiendo una persona (T), devolver la composición de su nombre completo (nombre y apellidos) (R: cadena de texto).

```
Function<Persona, String> nombreCompleto = p -> p.getNombre() + " " + p.getApellidos();
String nombreApellidos = nombreCompleto.apply(personas.get(0));

System.out.println(nombreApellidos);
```

En ambos casos, la ejecución de la línea `System.out.println(nombreApellidos)` mostrará por pantalla lo siguiente:

```
Paco Gento
```

5. Predicado. Predicate<T>

Es una interfaz funcional que **recibe un valor y devuelve un valor booleano**. Para crear un predicado debemos implementar la interfaz `Predicate<T>`, donde `T` indica el tipo del dato de entrada. Para ejecutar la función debemos hacer uso de su método **`test()`**.

Esta interfaz funcional es **enormemente útil para aplicar filtrado a los datos**, tal y como trabajaremos más adelante.

5.1 Implementación mediante clase

Supongamos que dada una Persona, queremos validar si es mayor de edad o no, es decir, recibimos una Persona y devolvemos un booleano. Un ejemplo es el siguiente.

```
public class FiltroMayorEdad implements Predicate<Persona> {

	@Override
	public boolean test(Persona p) {
		return p.edad() >= 18;
	}
}
```

También podríamos hacer que el filtro fuese personalizado a la edad que deseemos pasándosela al constructor del propio predicado. Podría ser de la siguiente forma.

```
public class FiltroPorEdad implements Predicate<Persona> {

	private int edad;


	public FiltroPorEdad(int edad) {
		this.edad = edad;
	}

	private int getEdad() {
		return edad;
	}

	@Override
	public boolean test(Persona p) {
		return p.edad() >= getEdad();
	}
}
```

Para hacer uso del predicado creado podemos implementar lo siguiente:

```
Predicate<Persona> filtroMayorEdad = new FiltroMayorEdad();

boolean mayorDeEdad = filtroMayorEdad.test(personas.get(0));
```

5.2 Implementación sin uso de clase independiente

A continuación vamos a ver un ejemplo de uso en el que implementamos un predicado, haciendo uso de una expresión lambda, para saber si la primera `Persona` de nuestra lista es mayor de edad.

```
Predicate<Persona> predicadoMayorDeEdad = p -> p.edad() >= 18;
boolean mayorDeEdad = predicadoMayorDeEdad.test(personas.get(0));
```

Para ir viendo la utilidad futura que podemos obtener a través de las interfaces funcionales (y su implementación mediante expresiones lambda) supongamos que tenemos una situación en la que recibimos por parámetro una colección de `Persona` y se desea eliminar todas aquellas personas que sean mayores de edad, es decir, quedarnos únicamente con los personas menores de edad. En un esquema puro de Programación Orientada a Objetos (POO) podemos implementar dicha funcionalidad de la siguiente forma:

```
private static void eliminarMayoresDeEdad(Collection<Persona> personas) {
	for(Persona persona : personas) {
		if(persona.edad() >= 18) {
			personas.remove(persona);
		}
	}
}
```

Con una expresión lambda, esa misma funcionalidad se podría implementar de la siguiente forma en la que el método removeIf recibe un Predicate por parámetro (el cual puede estar implementado como clase independiente o en expresión lambda).

```
private static void eliminarMayoresDeEdadExpresionLambda(Collection<Persona> personas) {
	personas.removeIf(persona -> persona.edad() >= 18);
}
```

6. Consumidor. Consumer<T>

Esta interfaz funcional **recibe un valor y NO devuelve nada**. Para crear un consumidor debemos implementar la interfaz `Consumer<T>`, donde `T` indica el tipo del dato de entrada. Para ejecutar la función debemos hacer uso de su método **`accept()`**.

Esta interfaz funcional es **enormemente útil para realizar acciones o procesos con un objeto o varios a la vez**, tal y como trabajaremos más adelante.

6.1 Implementación mediante clase

Supongamos que queremos crear un consumidor que dada una Persona, nos muestre la información de la misma.

```
public class PersonaPresentarsePorConsola implements Consumer<Persona> {
	@Override
	public void accept(Persona p) {
		System.out.println(p);
	}
}
```

Para hacer uso del consumidor podemos implementar el siguiente código fuente.

```
Consumer<Persona> consumidor = new PersonaPresentarsePorConsola();
consumidor.accept(personas.get(0));
personas.forEach(consumidor); // Se ejecuta sobre todas las personas de la lista
```

6.2 Implementación sin uso de clase independiente

A continuación vamos a ver un ejemplo de creación y uso de un consumidor (sin tener que crear una clase) en el que pasamos una Persona y nos muestra la información de dicha persona.

```
Consumer<Persona> presentarsePorConsola = p -> System.out.println(p);
presentarsePorConsola.accept(personas.get(0));
personas.forEach(presentarsePorConsola); // Se ejecuta sobre todas las personas
```

7. Proveedor. Supplier<R>

Es una interfaz funcional que **NO recibe ningún parámetro y devuelve un elemento**. Debemos hacer uso de la interfaz `Supplier<R>`, donde `R` indica el tipo devuelto, para implementarlo. Para ejecutar el proveedor debemos hacer uso de su método **`get()`**.

7.1 Implementación mediante clase

Si queremos generar un proveedor que nos genere una Persona con datos por defecto, los cuales después poder personalizar podemos crear un proveedor de la siguiente forma. En este caso, lo que nos interesa es generar una Persona con DNI de valor 0, nombre Anónimo y fecha de nacimiento del día de creación del objeto (de la Persona).

```
public class Nacimiento implements Supplier<Persona> {
	@Override
	public Persona get() {
		return new Persona(0, "Anónimo", "¿?", LocalDate.now());
	}
}
```

Para hacer uso del proveedor, podemos implementar el siguiente código fuente.

```
Supplier<Persona> nacimiento = new Nacimiento();
System.out.println(nacimiento.get());
```

7.2 Implementación sin uso de clase independiente

El mismo proveedor anterior lo podemos también implementar directamente, cuando lo necesitemos, sin tener que crear una clase.

```
Supplier<Persona> proveedorPersona = () -> new Persona(0, "Anónimo", "¿?", LocalDate.now());
Persona p = proveedorPersona.get();

System.out.println(p);
```

En ambos casos, obtendremos el mismo resultado:

```
Persona [dni=0, nombre=Anónimo, apellidos=¿?, fechaNacimiento=2022-12-28]
```

8. Comparadores haciendo uso de lambdas

Como acabamos de darnos cuenta, Java 8 nos trajo una gran variedad de herramientas, muy útiles, para el manejo de la información. Haciendo uso de expresiones lambda, podemos implementar un comparador, `Comparator`, de una forma muy directa y concreta.

Por ejemplo, si deseamos crear un comparador de personas por fecha de nacimiento podemos implementar el siguiente código:

```
Comparator<Persona> comparadorFechaNacimiento =
                    (p1, p2) -> p1.getFechaNacimiento().compareTo(p2.getFechaNacimiento());
```

La explicación de lo implementado es la siguiente:

- `p1` y `p2` son la referencia a los dos objetos Persona que se van a comparar.
- Mediante el método `getFechaNacimiento()` obtenemos la fecha de nacimiento de ambos objetos que se están comparando.
- Mediante el método `compareTo()`, comparamos ambas fechas de nacimiento.

Piensa. Si utilizamos el anterior comparador, ¿cómo nos va a ordenar a las personas: de mayor a menor edad o de menor a mayor edad?

9. Métodos útiles

A continuación, podemos ver una serie de métodos que nos van a ser útiles si utilizamos expresiones lambda.

Método	Ejemplo
*Iterable.**forEach**(Consumer)*	`personas.forEach(p -> System.out.println(p));`
*Collection.**removeIf**(Predicate)*	`personas.removeIf(p -> p.edad() < 18);`
*List.**sort**(Comparator)*	`personas.sort((p1,p2) -> Integer.compare(p1.edad(), p2.edad()));`

CC BY NC SA

10. Buenas prácticas

A continuación, se indican una serie de buenas prácticas que nos permitirán implementar un código más legible y más fácil de mantener a la hora de usar expresiones lambda.

1) **Las expresiones lambda deben ser breves y autoexplicativas**. Si es posible, deberíamos usar construcciones de una línea en lugar de un gran bloque de código. No afectará al rendimiento pero sí a la legibilidad. Debemos tener en mente que deben ser una expresión, no una narración, expresando de forma concisa la funcionalidad que realizan. Es importante tener en cuenta que no debemos usar esta regla "*lambda de una línea*" como dogma.

2) **Evitar especificar tipos de parámetros**. Un compilador, en la mayoría de los casos, puede resolver el tipo de parámetros lambda con la ayuda de la inferencia de tipos. En consecuencia, agregar un tipo a los parámetros es opcional y se puede omitir.

Podemos hacer esto:

```
(a, b) -> a.toLowerCase() + b.toLowerCase();
```

En lugar de esto:

```
(String a, String b) -> a.toLowerCase() + b.toLowerCase();
```

3) **Evitar los paréntesis alrededor de un único parámetro**. La sintaxis de las expresiones lambda únicamente requiere paréntesis alrededor de más de un parámetro, o cuando no hay ningún parámetro.

Podemos hacer esto:

```
a -> a.toLowerCase();
```

En lugar de esto:

```
(a) -> a.toLowerCase();
```

4) **Evitar declaraciones de devolución y llaves**. Las llaves y declaraciones de retorno son opcionales en cuerpos lambda de una línea. Esto significa que se pueden omitir para mayor claridad y concisión.

Podemos hacer esto:

```
a -> a.toLowerCase();
```

En lugar de esto:

```
a -> {return a.toLowerCase()};
```

5) **Usar referencias de métodos**. En muchas ocasiones, las expresiones lambda llaman a métodos que ya están implementados en otros lugares. En esta situación, es muy útil utilizar otra función de Java 8, las referencias a métodos. Esto no siempre es más corto, pero hace que el código sea más legible.

En lugar de hacer lo siguiente:

```
a -> a.toLowerCase();
```

Podríamos sustituirlo por:

```
String::toLowerCase;
```

11. Aspectos a tener en cuenta

Las expresiones lambda de Java 8 nos ofrecen varias mejoras con respecto a las versiones anteriores.

- Nos permiten hacer uso de las herramientas del paradigma de programación funcional, el cual es otro enfoque diferente a la programación orientada a objetos.
- Hacen nuestro código más preciso y legible, mejorando, en consecuencia, su mantenibilidad.
- Su utilización junto con la **API Stream** hace más fácil la ejecución concurrente de tareas.